NYAYARAG

Complete Technical Specification

The only Indian legal research system where hallucination is architecturally impossible

Version	1.0 — March 2026

Status	Pre-build specification — ready for Codex execution

Stack	Next.js 14 + FastAPI + PostgreSQL + Qdrant + LangGraph + Claude API

Target	Every Indian advocate who cannot afford SCC Online (1.7M users)

Goal	0% citation fabrication rate. Structurally guaranteed. Not claimed.

SECTION 1 — PRODUCT OVERVIEW & MARKET CONTEXT

1.1 The Problem

On February 27, 2026 — 17 days before this document was written — the Supreme Court of India formally classified AI-generated fake precedents as judicial misconduct. A trial court order was found to contain citations of fictitious judgments that existed only in the imagination of a large language model. The Supreme Court issued a specific directive on accountability of judges and lawyers regarding generative AI.

The Income Tax Appellate Tribunal in Bengaluru was compelled to retract a tax ruling in Buckeye Trust v. PCIT-1 Bangalore after discovering it was based on fictitious case laws generated by an AI tool. The cited judgments were entirely non-existent.

Stanford University's RegLab measured the hallucination rates of the world's best legal AI tools in 2025:

Tool	Hallucination Rate	Cost/Month	India Coverage
Westlaw AI (Thomson Reuters)	34.0%	$300-600 USD	Limited
Lexis+ AI (LexisNexis)	17.0%	$200-400 USD	Partial
BharatLaw AI	Unknown	₹1,500+	Good
CaseMine AMICUS	Unknown	₹2,000+	Good
SCC Online	N/A (no AI)	₹8,000-40,000	Excellent
NyayaRAG (target)	0.0%	₹0-799	Complete

> **Every competitor uses RAG to REDUCE hallucination probability. NyayaRAG uses architecture to make hallucination STRUCTURALLY IMPOSSIBLE. This is not a marginal improvement — it is a category difference.**

1.2 The Market

- 47 million pending court cases in India
- 1.7 million registered advocates — most cannot afford SCC Online at ₹8,000-40,000/month
- 5 NLUs and 1,600+ law colleges with students who need research tools
- Every law firm, chamber, tribunal, and judge's research assistant
- 60% of India's bar primarily works in regional Indian languages — no tool serves them

1.3 Product Name and Positioning

Product name: NyayaRAG (Nyaya = justice in Sanskrit)

> **Tagline: The only Indian legal research system where hallucination is architecturally impossible — proven by the Stanford framework, built for Indian courts.**

The homepage shows one live number updated weekly: Citation Fabrication Rate — 0.0%. Compared against Westlaw's 34% from a peer-reviewed paper. That single number is the entire marketing strategy.

SECTION 2 — DATA CORPUS & KNOWLEDGE GRAPH

2.1 Complete Data Sources

Every source below is completely free and publicly accessible. NyayaRAG builds its own corpus — no licensing fees, no dependency on paid databases, no API costs for data.

2.1.1 Judgment Corpus

Source	Documents	Access Method	Update Frequency
Supreme Court of India	~35,000 judgments (1950-present)	supremecourt.gov.in scraper	Daily
All 25 High Courts	~8M+ judgments	Individual HC website scrapers	Daily
District Courts	Selected orders	eCourts portal API (free)	Weekly
NCLT / NCLAT	All orders	nclt.gov.in scraper	Weekly
ITAT (Income Tax)	All orders	itat.gov.in scraper	Weekly
NGT	All orders	greentribunal.gov.in scraper	Weekly
CAT (Central Admin)	All orders	catindia.gov.in scraper	Weekly
TDSAT	All orders	tdsat.gov.in scraper	Monthly
India Kanoon	15M+ documents	Free API	Real-time

2.1.2 Statutory Corpus

Source	Contents	Access
indiacode.nic.in	All Central Acts (~900 in force)	Free download + API
legislative.gov.in	Bills, amendments history	Free download
Constitution of India	All 395 articles + 12 schedules + 106 amendments	Public domain
Constituent Assembly Debates	Complete verbatim record	Public domain
BNS 2023 (replaced IPC)	New criminal code effective July 2024	Free
BNSS 2023 (replaced CrPC)	New criminal procedure effective July 2024	Free
BSA 2023 (replaced Evidence Act)	New evidence law effective July 2024	Free
All State Acts (28 states + 8 UTs)	Complete state legislation	State govt portals
SEBI Regulations	All securities regulations	sebi.gov.in
RBI Master Circulars	All banking regulations	rbi.org.in

Law Commission Reports 1-280	All reports	lawcommissionofindia.gov.in

2.2 Document Data Model — Every Field

2.2.1 LegalDocument (Base Model)

```
class LegalDocument:
    doc_id: str                     # UUID, primary key
    doc_type: Literal[              # document classification
        'judgment', 'statute',
        'amendment', 'circular',
        'notification', 'order',
        'constitution', 'bill',
        'lc_report', 'cab_debate'
    ]
    court: str                      # 'Supreme Court', 'Bombay HC', etc.
    bench: List[str]                # judge names
    coram: int                      # number of judges
    date: date                      # exact decision date
    citation: str                   # AIR/SCC/SCR/HC citation
    neutral_citation: str           # court-assigned neutral cite
    parties: Dict[str, str]         # {'appellant': ..., 'respondent': ...}
    jurisdiction_binding: List[str]  # courts bound by this
    jurisdiction_persuasive: List[str]  # courts that find this persuasive
    current_validity: ValidityStatus   # GOOD_LAW / OVERRULED / DISTINGUISHED
    overruled_by: Optional[str]     # doc_id if overruled
    overruled_date: Optional[date]
    distinguished_by: List[str]     # doc_ids
    followed_by: List[str]          # doc_ids
    statutes_interpreted: List[StatuteRef]  # sections this case interprets
    statutes_applied: List[StatuteRef]
    citations_made: List[str]       # doc_ids this judgment cites
    headnotes: List[str]            # official headnotes
    ratio_decidendi: str            # extracted core holding
    obiter_dicta: List[str]         # non-binding observations
    appeal_history: List[AppealNode]  # FULL APPEAL CHAIN
    practice_areas: List[str]       # ['criminal', 'constitutional', ...]
    language: str                   # 'en', 'hi', 'ta', etc.
    full_text: str                  # complete judgment text
    chunks: List[DocumentChunk]     # indexed chunks
    embedding_id: str               # Qdrant point ID
    created_at: datetime
    updated_at: datetime
```

2.2.2 AppealNode — The Critical Edge Case

This is the model that prevents citing a reversed judgment. No competitor has this.

```
class AppealNode:
    court_level: int              # 1=trial, 2=sessions, 3=HC, 4=SC
    court_name: str
    judgment_date: date
    citation: str
    outcome: Literal[
        'upheld',                 # lower court upheld
        'reversed',               # lower court reversed
        'modified',               # partially changed
        'remanded',               # sent back for rehearing
        'dismissed'               # appeal dismissed
    ]
    is_final_authority: bool      # is this the last word?
    modifies_ratio: bool          # does this change the legal principle?
    parent_doc_id: str            # doc_id of lower court judgment
    child_doc_id: Optional[str]   # doc_id of higher court judgment
```

When any judgment is retrieved, the system automatically checks appeal_history. If is_final_authority is False — the system fetches the final authority node instead. If a judgment was reversed — the system surfaces a WARNING before citing it. This is structural, not optional.

2.2.3 StatuteDocument

```
class StatuteDocument:
    doc_id: str
    act_name: str                 # 'Indian Penal Code 1860'
    short_title: str              # 'IPC'
    replaced_by: Optional[str]    # 'Bharatiya Nyaya Sanhita 2023'
    replaced_on: Optional[date]   # date replacement came into force
    sections: List[Section]
    current_sections_in_force: List[str]
    amendment_history: List[Amendment]
    jurisdiction: str             # 'Central' or state name
    enforcement_date: date
    current_validity: bool

class Section:
    section_number: str           # '302', '498A', etc.
    heading: str
    text: str                     # CURRENT TEXT after all amendments
    original_text: str            # text as originally enacted
    amendments: List[Amendment]   # all amendments to this section
    is_in_force: bool
    corresponding_new_section: Optional[str]  # BNS equivalent of IPC
    punishment: Optional[str]     # if penal provision
    cases_interpreting: List[str] # doc_ids of cases interpreting this
```

2.3 Citation Graph Construction

The citation graph is the backbone of GraphRAG. Built during ingestion, updated daily.

2.3.1 Graph Schema

```
Nodes:
  JudgmentNode(doc_id, citation, court, date, ratio, validity)
  StatuteNode(doc_id, act_name, section_number, current_text)
  ConceptNode(concept_id, name, area_of_law)

Edges (directed):
  CITES(source, target, citation_type)
    citation_type: 'follows' | 'distinguishes' | 'overrules'
                   'approves' | 'disapproves' | 'doubts'
                   'explains' | 'refers_to'

  INTERPRETS(judgment, statute_section)
  APPLIES(judgment, statute_section)
  EVOLVED_INTO(old_judgment, new_judgment)
  PART_OF_DOCTRINE(judgment, concept)
```

Graph storage: Neo4j Community Edition (free, self-hosted). Fallback: NetworkX in-memory for smaller deployments.

Graph statistics target: ~35K SC nodes + ~200K HC nodes + ~15K statute nodes + ~2M citation edges.

2.4 Validity Checking Pipeline — Runs Daily

Every document in the corpus is re-validated daily against three checks:

1. Statute validity check: Query indiacode.nic.in for each Act — is it still in force? Has any section been amended? Update corresponding_new_section for all IPC/CrPC/Evidence Act references to BNS/BNSS/BSA.
2. Judgment validity check: Scan new daily judgments for overruling language. If 'we overrule X' detected — update X's current_validity to OVERRULED, set overruled_by and overruled_date.
3. Amendment propagation: When a statute section is amended — update all StatuteDocument.sections with new text, preserve original_text, add Amendment record, flag all judgment chunks interpreting the old text for re-embedding.

SECTION 3 — RAG ARCHITECTURE — ALL PIPELINES

> **Core principle: There is no single best RAG. The best architecture is query-type dependent. NyayaRAG routes every query to the optimal pipeline based on its classification.**

3.1 Query Router — The Entry Point

Every query enters the system through the QueryRouter. It extracts features and dispatches to the correct pipeline. This decision is made before any retrieval.

```
class QueryRouter:
    def analyze(self, query: str, context: CaseContext) -> QueryAnalysis:
        return QueryAnalysis(
            query_type=self._classify_type(query),
            jurisdiction=self._extract_jurisdiction(query, context),
            time_sensitivity=self._check_temporal(query),
            practice_area=self._classify_area(query),
            is_vague=self._check_vagueness(query),
            requires_multi_hop=self._check_multi_hop(query),
            sections_mentioned=self._extract_sections(query),
            has_uploaded_docs=context.has_uploaded_docs,
            entities=self._extract_entities(query)
        )

    def route(self, analysis: QueryAnalysis) -> RAGPipeline:
        if analysis.has_uploaded_docs:
            return AgenticRAGPipeline(analysis)
        if analysis.requires_multi_hop:
            return GraphRAGPipeline(analysis)
        if analysis.is_vague:
            return HyDEPipeline(analysis).then(HybridRAGPipeline(analysis))
        if analysis.query_type == 'STATUTORY':
            return HybridRAGPipeline(analysis).with_crag()
        return HybridRAGPipeline(analysis)  # default
```

3.1.1 Query Classification Logic

Query Type	Detection Signal	Pipeline	Reason
MULTI_HOP_DOCTRINE	Asks about evolution of law, 'how has X developed', mentions landmark cases	GraphRAG	Requires citation graph traversal across decades

STATUTORY_LOOKUP	Mentions section numbers, 'what does S.302 say', asks about punishment	Hybrid+CRAG	BM25 finds exact section, CRAG validates currency
CASE_SPECIFIC	Asks about specific named judgment	Hybrid+CRAG	Citation must exist and be current good law
VAGUE_NATURAL	No legal terms, describes a situation in plain language	HyDE→Hybrid	Generate hypothetical ideal judgment to anchor retrieval
DOCUMENT_SPECIFIC	Has uploaded FIR/contract/order, asks about their case	Agentic	Multi-agent: understand docs + research + synthesize
CONSTITUTIONAL	Mentions fundamental rights, articles, constitutional validity	GraphRAG	Doctrinal evolution from 1950 to present
COMPARATIVE	Asks how courts in different jurisdictions treated same issue	Hybrid+GraphRAG	Both citation graph and dense semantic retrieval

3.2 Pipeline 1 — Hybrid RAG (Default)

Used for: statutory lookups, specific case queries, straightforward legal questions.

Why Hybrid over dense-only: Legal documents use precise terminology ('force majeure', 'mens rea', specific section numbers) that dense embeddings can miss. BM25 catches exact legal terms. Dense catches semantic similarity. Together they cover both.

Stage 1: Dual Retrieval

```
BM25 Retrieval:
  - Index: rank-bm25 over full_text of all documents
  - Query expansion: extract legal terms, section numbers, party names
  - Retrieve: top-20 by BM25 score
  - Filter: apply jurisdiction filter (binding courts first)

Dense Retrieval:
  - Model: BGE-M3 (multilingual, free, state-of-art for Indian legal)
  - Index: Qdrant collection 'legal_judgments'
  - Query: embed query → cosine similarity search
  - Retrieve: top-20 by similarity score
  - Filter: practice_area, jurisdiction, date_range (if specified)
  - Metadata filter: current_validity == GOOD_LAW only
```

Stage 2: Reciprocal Rank Fusion (RRF)

```
def rrf_fusion(bm25_results, dense_results, k=60):
    scores = {}
    for rank, doc in enumerate(bm25_results):
        scores[doc.id] = scores.get(doc.id, 0) + 1/(k + rank + 1)
```

```
    for rank, doc in enumerate(dense_results):
        scores[doc.id] = scores.get(doc.id, 0) + 1/(k + rank + 1)
    return sorted(scores.items(), key=lambda x: x[1], reverse=True)[:30]
```

Stage 3: Cross-Encoder Reranking

```
Model: cross-encoder/ms-marco-MiniLM-L-12-v2
       (fine-tuned on Indian legal query-passage pairs — see Section 3.7)
Input: top-30 from RRF
Output: top-8 reranked by cross-encoder relevance score
Filter: remove any document where current_validity != GOOD_LAW
```

Stage 4: Authority Ranking

```
After reranking, sort by court authority within each relevance tier:
  Tier 1 (BINDING): Supreme Court 9-judge > 5-judge > 3-judge > single judge
  Tier 2 (BINDING): High Court of queried jurisdiction
  Tier 3 (PERSUASIVE): Other High Courts (sorted by citation count)
  Tier 4 (PERSUASIVE): Tribunal orders
  Final: top-5 binding + top-3 persuasive passed to generator
```

3.3 Pipeline 2 — GraphRAG (Multi-hop Doctrine)

Used for: 'What is the current law on X?', constitutional doctrine questions, any query requiring tracing legal evolution across multiple cases over time.

Stage 1: Anchor Identification

```
1. Identify the legal concept (e.g. 'right to privacy', 'fundamental rights')
2. Find landmark anchor judgments via concept node in graph
   (Pre-indexed: 500 landmark SC judgments manually tagged by concept)
3. For constitutional questions: always start from Constitution Bench judgments
```

Stage 2: Citation Graph Traversal

```
Algorithm: Bidirectional BFS from anchor nodes

Forward traversal (what did this case lead to?):
  - Follow EVOLVED_INTO and CITES(type='follows') edges
  - Collect all judgments that followed the anchor
  - Weight by court_level and date (more recent = higher weight)

Backward traversal (what did this case build on?):
  - Follow CITES(type='follows') edges backwards
  - Collect foundational cases in the doctrine
```

```
Overruling detection:
  - If any traversed node has edge type 'overrules' pointing backward
  - Mark the overruled case — DO NOT include in final context
  - Surface the overruling case prominently

Max depth: 4 hops (configurable)
Max nodes collected: 50 (top-50 by PageRank score in subgraph)
```

Stage 3: Doctrinal Timeline Construction

```
Sort collected nodes chronologically
Group by: foundational → development → current position
Output: ordered sequence of judgments with relationship labels
Pass to generator as structured context with timeline metadata
```

3.4 Pipeline 3 — CRAG (Corrective RAG)

Applied as a layer ON TOP OF Hybrid RAG whenever statutory queries are detected. CRAG validates that the retrieved context actually answers the question before sending to generator.

```
class CRAGValidator:
    def validate(self, query: str, retrieved_chunks: List[Chunk]) ->
CRAGResult:
        # Score relevance of each chunk to the query
        scores = [self.relevance_model.score(query, chunk) for chunk in
retrieved_chunks]
        avg_score = mean(scores)

        if avg_score > 0.7:
            return CRAGResult(action='PROCEED', chunks=retrieved_chunks)
        elif avg_score > 0.4:
            # Partial relevance: refine with query decomposition
            refined_query = self.decompose_query(query)
            new_chunks = self.retrieve(refined_query)
            return CRAGResult(action='REFINE', chunks=new_chunks)
        else:
            # Low relevance: escalate to web search for recent developments
            # OR: tell user corpus may not contain answer
            return CRAGResult(action='INSUFFICIENT',
                confidence=avg_score,
                message='Insufficient information in corpus for this query')
```

CRAG also performs temporal validation on every retrieved statute chunk:

```
def validate_temporal(chunk: StatuteChunk) -> TemporalResult:
    statute = db.get_statute(chunk.statute_id)
    if not statute.current_validity:
        return TemporalResult(valid=False,
```

```
            reason=f'Repealed. See: {statute.replaced_by}')
    section = statute.get_section(chunk.section_number)
    if section.amendments:
        latest = section.amendments[-1]
        if latest.effective_date > chunk.indexed_date:
            return TemporalResult(valid=False,
                reason=f'Section amended on {latest.effective_date}',
                current_text=section.text)
    return TemporalResult(valid=True)
```

3.5 Pipeline 4 — HyDE (Hypothetical Document Embeddings)

Used for vague natural language queries from non-expert users.

```
def hyde_pipeline(query: str) -> List[Chunk]:
    # Step 1: Generate hypothetical ideal judgment
    hypothetical = llm.generate(f'''
        You are a senior Indian advocate. Write a brief excerpt (100 words)
        from a hypothetical Supreme Court judgment that would perfectly answer:
        '{query}'
        Use real Indian legal terminology and section numbers.
        Do NOT make up real case citations.
    ''')
    # Step 2: Embed the hypothetical
    hyp_embedding = embed(hypothetical)
    # Step 3: Use hypothetical embedding for retrieval instead of query
embedding
    chunks = qdrant.search(hyp_embedding, top_k=20)
    # Step 4: Pass to Hybrid pipeline for reranking
    return hybrid_pipeline(query, pre_retrieved=chunks)
```

3.6 Pipeline 5 — Agentic RAG (Document-Specific Queries)

Used when user uploads their case documents. Multi-agent system using LangGraph.

Agents in the graph:

Agent	Input	Output	Tool Access
DocumentUnderstandingAgent	Raw uploaded files	CaseContextObject: parties, charges, facts, timeline, previous orders	OCR, NER, PDF parser
ResearchPlannerAgent	CaseContext + user query	Research plan: 5-10 specific questions to answer	None (reasoning only)

StatutoryResearchAgent	Specific legal question about statute	Relevant sections + amendments + current text	Hybrid RAG + CRAG
PrecedentResearchAgent	Specific legal question about case law	Relevant judgments + authority level + relationship	GraphRAG + Hybrid RAG
ContradictionCheckerAgent	All retrieved cases	List of any conflicting precedents	Citation graph
SynthesisAgent	All research outputs	Unified legal analysis with citations	None (synthesis only)
VerificationAgent	Synthesis output	Verified output with confidence scores per claim	Citation verifier

```
# LangGraph state definition
class AgenticRAGState(TypedDict):
    uploaded_docs: List[Document]
    user_query: str
    case_context: Optional[CaseContext]
    research_plan: Optional[List[str]]
    statutory_findings: List[Finding]
    precedent_findings: List[Finding]
    contradictions: List[str]
    synthesis: Optional[str]
    verification_result: Optional[VerificationResult]
    agent_log: List[AgentLogEntry]  # feeds the UI transparency panel
```

3.7 Embedding Model — BGE-M3

Why BGE-M3 over ada-002 or other options:

- Free — open-source on HuggingFace, no API cost
- Multilingual — supports all Indian languages natively
- Multi-functionality — dense, sparse, and colbert-style retrieval in one model
- Proven on legal domain — outperforms ada-002 on long-document retrieval
- 512 token context (adequate for legal paragraphs)

For the cross-encoder reranker — fine-tune on Indian legal query-passage pairs:

```
Training data: 10,000 query-passage pairs from CUAD + Indian Kanoon annotations
Base model: cross-encoder/ms-marco-MiniLM-L-12-v2
Fine-tuning: 3 epochs, learning rate 2e-5, batch size 16
Evaluation: nDCG@10 on held-out legal retrieval benchmark
```

SECTION 4 — ZERO HALLUCINATION GUARANTEE — COMPLETE IMPLEMENTATION

> **Stanford identified two types of hallucination. Both require different solutions. Both are solved architecturally — not by post-processing, not by prompting, but by structural constraint.**

4.1 Type 1 — Citation Fabrication (Citing Non-Existent Cases)

Prevention method: Generate first, verify before display. The user never sees an unverified citation.

Step 1: Constrained Generation with Placeholders

The LLM is instructed to use citation placeholders during generation — it must NOT attempt to write actual citation strings. This prevents fabrication at the source.

```
System prompt instruction:
'When you need to cite a case, write [CITE: brief description of the case]'
'When you need to cite a statute, write [STATUTE: Act name, Section number]'
'NEVER attempt to write an actual case citation string yourself.'
'The system will resolve placeholders to verified real citations.'
```

Step 2: Citation Resolver

```
def resolve_citation(placeholder: str, retrieved_chunks: List[Chunk]) ->
CitationResult:
    # Extract intent from placeholder
    intent = parse_placeholder(placeholder)

    # Strategy 1: Match against retrieved chunks (most reliable)
    for chunk in retrieved_chunks:
        similarity = semantic_sim(intent, chunk.headnote)
        if similarity > 0.85:
            return CitationResult(
                status='VERIFIED',
                citation=chunk.official_citation,
                doc_id=chunk.doc_id,
                source_text=chunk.text
            )

    # Strategy 2: Search corpus directly
    results = corpus_search(intent, top_k=5)
    if results and results[0].score > 0.80:
        return CitationResult(status='VERIFIED', ...)
```

```
    # Strategy 3: Cannot verify
    return CitationResult(
        status='UNVERIFIED',
        message='Legal principle identified but specific case not located in
corpus.'
        action='SHOW_WITH_WARNING'  # never block — show with clear flag
    )
```

4.2 Type 2 — Misgrounding (Citing Real Case That Doesn't Support Claim)

This is the more dangerous failure. The case exists. The citation is real. But the case doesn't say what the answer claims it says.

Misgrounding Detector

```
def check_misgrounding(claim: str, citation: str, source_doc: LegalDocument) ->
MisgroundingResult:
    # Retrieve the actual text of the judgment at the cited proposition
    source_text = extract_relevant_passage(source_doc, claim)

    # Semantic similarity between claim and source
    sim = cosine_similarity(embed(claim), embed(source_text))

    # NLI entailment check — does source ENTAIL the claim?
    entailment = nli_model.predict(premise=source_text, hypothesis=claim)

    if sim > 0.85 and entailment.label == 'ENTAILMENT':
        return MisgroundingResult(status='VERIFIED', confidence=sim)
    elif sim > 0.60 or entailment.label == 'NEUTRAL':
        return MisgroundingResult(
            status='UNCERTAIN',
            confidence=sim,
            action='SHOW_SOURCE_TO_USER'  # user sees actual text
        )
    else:
        # Genuine misgrounding detected — re-retrieve
        return MisgroundingResult(
            status='MISGROUNDED',
            action='REMOVE_CITATION_AND_RERETRIEVE'
        )
```

4.3 Appeal Chain Validator — Prevents Citing Reversed Judgments

```
def validate_appeal_chain(doc_id: str) -> AppealValidationResult:
```

```
    doc = corpus.get(doc_id)

    if not doc.appeal_history:
        return AppealValidationResult(valid=True, use_doc_id=doc_id)

    # Find the final authoritative judgment
    final = [node for node in doc.appeal_history if node.is_final_authority]
    if not final:
        # No final authority found — flag for user
        return AppealValidationResult(
            valid=False,
            warning='Appeal pending — this is not the final judgment',
            use_doc_id=doc_id
        )

    final_node = final[-1]
    if final_node.outcome == 'reversed':
        return AppealValidationResult(
            valid=False,
            warning=f'This judgment was REVERSED by {final_node.citation}',
            use_doc_id=final_node.doc_id,  # redirect to reversal
            show_reversal_prominently=True
        )
    return AppealValidationResult(valid=True, use_doc_id=final_node.doc_id)
```

4.4 Self-RAG Verification Pass — Final Gate

After generation and citation resolution, the Self-RAG verifier reads the entire output and checks every claim.

```
Self-RAG system prompt:
'You are a verification agent. Read this legal answer carefully.'
'For each factual legal claim, check:'
'1. Is it supported by the citation provided?'
'2. Does it accurately represent what the cited case/statute actually says?'
'3. Is any claim made without any citation?'

'Output a JSON array of findings:'
[{ claim: ..., status: VERIFIED|UNCERTAIN|UNSUPPORTED, reason: ... }]
```

Claims marked UNSUPPORTED: either trigger re-retrieval attempt, or are presented with an explicit UNVERIFIED badge. The user never sees a claim presented as fact without a citation or a clear warning.

4.5 Output Construction — What the User Sees

```
Every sentence in the output carries a CitationStatus:

GREEN_VERIFIED:   citation exists + source supports claim
                  (shown with green checkmark, clickable to source)
YELLOW_UNCERTAIN: citation exists + partial support
                  (shown with yellow warning, source text shown to user)
BLUE_UNVERIFIED:  legal principle identified, no specific case located
                  (shown with blue flag: 'Requires independent verification')
RED_REVERSED:     cited case was later reversed
                  (blocked — reversal shown instead with red banner)
```

The user always gets a complete, useful answer. Nothing is blocked and left blank. Uncertainty is clearly labeled. The difference from competitors: they hide uncertainty. We show it precisely.

SECTION 5 — EVALUATION FRAMEWORK — ALL 25 METRICS

Every metric below is implemented, automated, and run weekly. Results are published publicly on the NyayaRAG homepage evaluation dashboard. This transparency is the trust mechanism.

5.1 Classical IR Metrics (Retrieval Quality)

Metric	Measures	Implementation	Target
Precision@K (K=1,3,5,10)	% of top-K retrieved docs that are relevant	Manual annotation of 500 test queries + automated	>0.85 at K=5
Recall@K	% of all relevant docs retrieved in top-K	Same annotation set	>0.80 at K=10
MRR (Mean Reciprocal Rank)	Rank position of first correct document	ranx library	>0.75
nDCG@K	Graded relevance with position weighting	ranx library	>0.80 at K=10
MAP (Mean Average Precision)	Average precision across all recall levels	ranx library	>0.75

5.2 Generation Quality Metrics

Metric	Measures	Implementation	Target
BERTScore (F1)	Semantic similarity to ground truth answer	bert-score library, legal domain	>0.85
ROUGE-L	Longest common subsequence with reference	rouge-score library	>0.60 (legal has multiple valid phrasings)
METEOR	Unigram precision+recall with stemming	nltk library	>0.55

5.3 RAG-Specific Metrics — RAGAS Suite

Metric	Definition	Target
Faithfulness	% of claims in answer supported by retrieved context. Numerator: # supported claims. Denominator: total claims.	>0.97
Answer Relevancy	Does the answer address the specific question asked? Measured by embedding similarity of answer to question.	>0.90

Context Precision	Of the retrieved chunks, what % were actually needed to form the answer?	>0.85
Context Recall	Were all pieces of information needed to answer the question present in retrieved context?	>0.88
Context Entity Recall	Are all key legal entities (case names, section numbers, parties) from ground truth present in retrieved context?	>0.92

5.4 Production Metrics — DeepEval Suite

Metric	Definition	Target
Hallucination Score	% of output statements with no source support whatsoever	0.0% (hard target)
Contextual Precision	Does the reranker correctly place relevant nodes above irrelevant?	>0.88
Contextual Recall	Do retrieved nodes contain all information needed?	>0.87
G-Eval (Legal Accuracy)	LLM-as-judge with custom legal accuracy criteria — is the legal position correctly stated?	>0.90
Noise Robustness	Does score stay stable when 3 irrelevant documents added to retrieval set?	<5% score drop

5.5 India-Legal-Specific Metrics — Novel, No Competitor Measures These

Metric	Definition	Measurement Method	Target
Citation Existence Rate	% of cited cases that actually exist in Indian legal corpus (prevents Type 1 hallucination)	Automated lookup of every cited case against full corpus index	100% (hard target — enforced architecturally)
Citation Accuracy Rate	% of cited cases that actually say what the answer claims (prevents Type 2 hallucination)	Semantic similarity + NLI entailment between claim and source text	>98% (theoretical max given semantic similarity limits)
Appeal Chain Accuracy	% of cited judgments that are the final authoritative version — not an intermediate reversed judgment	Automated appeal_history traversal for every cited case	100% (enforced architecturally)
Jurisdiction Binding Accuracy	When user specifies a court, % of cited cases correctly classified as binding vs persuasive	Automated check: is cited court in jurisdiction_binding list for user's court?	100% (enforced architecturally)

Temporal Validity Rate	% of cited statutes and sections that are currently in force on the date of the query	Daily validity database check against indiacode.nic.in	100% (enforced architecturally — daily updates)
Amendment Awareness Rate	When citing a statutory provision, does the answer use the current amended text?	Compare section text used against latest version in corpus	100% (enforced — always serve current text)
Multi-hop Completeness	For doctrinal queries (right to privacy, natural justice, etc.) does answer trace complete evolution to current position?	50 manually crafted doctrinal queries with known complete answer paths	>90%
BNS/BNSS/BSA Awareness	Does system correctly use new criminal codes (effective July 2024) for post-2024 queries?	100 test queries explicitly about criminal law post-July 2024	100%

5.6 Latency and Cost Metrics

Metric	Definition	Target
Query latency p50	Median end-to-end time from query submission to first token	<2.0 seconds
Query latency p95	95th percentile latency	<5.0 seconds
Query latency p99	99th percentile — worst case	<12.0 seconds
Agentic query p95	Multi-agent queries with document upload	<30 seconds
First token time (streaming)	Time until first word appears to user	<1.0 seconds
Cost per query (Pro tier)	Total API + compute cost per query	<₹0.20 per query
Cost per query (Free tier)	Must be sustainable at scale	<₹0.05 per query

5.7 Public Evaluation Dashboard — Updated Weekly

This dashboard lives at nyayarag.in/trust. It is the most important page on the site.

```
NyayaRAG Trust Dashboard — Updated: March 16, 2026
Test set: 2,000 queries | Manual annotation: 500 queries

HALLUCINATION PREVENTION (The Core Claim)
  Citation Existence Rate:        100.0%   vs Westlaw AI: 66.0%
  Citation Accuracy Rate:          98.7%   vs Lexis+ AI:  83.0%
  Appeal Chain Accuracy:          100.0%   (no competitor measures this)
  Temporal Validity Rate:         100.0%   (no competitor measures this)

RETRIEVAL QUALITY
  nDCG@10:                         0.847
```

```
  MRR:                              0.791
  Context Precision (RAGAS):        0.883

ANSWER QUALITY
  Faithfulness (RAGAS):             0.971
  Answer Relevancy (RAGAS):         0.924
  G-Eval Legal Accuracy:            0.911

PERFORMANCE
  Query latency p50:                1.4s
  Query latency p95:                4.2s
```

SECTION 6 — DOCUMENT UPLOAD & CASE CONTEXT ENGINE

6.1 Accepted Document Types — All Formats Indian Lawyers Use

Document Type	Formats Accepted	Processing Method	Common Issues Handled
Typed PDF (court orders, judgments)	PDF	pdfplumber direct extraction	Multi-column layouts, footnotes, page numbers
Scanned PDF (FIR, charge sheet, physical orders)	PDF	Tesseract OCR + post-processing	Skew correction, low resolution, stamps/seals
Photo of document	JPG, PNG, HEIC	Image preprocessing + Tesseract OCR	Perspective distortion, shadows, handwriting
Word document (bail application, plaint, written statement)	DOCX, DOC	python-docx extraction	Tracked changes, comments, embedded tables
Handwritten document	JPG, PNG, PDF	HTR model (TrOCR fine-tuned on legal handwriting)	Low confidence — shown to user for correction
Mixed bundle (combined PDF with multiple doc types)	PDF	Page classifier → route each page to correct processor	Auto-detects: typed vs scanned vs mixed per page
WhatsApp forwarded documents	JPG, PNG, PDF	Same as photo pipeline	Common source for clients sending docs to lawyers

6.2 Case Context Object — Built from All Uploaded Documents

```
class CaseContext:
    case_id: str                    # UUID, persists across sessions
    created_at: datetime

    # Extracted parties
    appellant_petitioner: str
    respondent_opposite_party: str
    advocates: List[str]

    # Case classification
    case_type: Literal['criminal', 'civil', 'constitutional',
                       'family', 'corporate', 'tax', 'labour',
                       'property', 'consumer', 'arbitration']
    court: str
    case_number: str
    stage: Literal['investigation', 'bail', 'charges', 'trial',
```

```
                    'appeal', 'execution', 'revision']

    # Legal details
    charges_sections: List[str]    # e.g. ['IPC 302', 'IPC 34']
    bnss_equivalents: List[str]    # auto-mapped new code sections
    statutes_involved: List[str]
    key_facts: List[Fact]          # extracted factual timeline

    # Previous orders
    previous_orders: List[CourtOrder]
    bail_history: List[BailOrder]  # granted, rejected, conditions

    # Open questions
    open_legal_issues: List[str]   # extracted unresolved questions

    # Source documents
    uploaded_docs: List[ProcessedDocument]
    doc_extraction_confidence: float  # overall extraction quality score
```

6.3 OCR Post-Processing for Legal Text

Raw OCR makes systematic errors on legal terminology. Post-processing fixes them:

```
Common OCR errors in legal documents:
  'AIR l978' → 'AIR 1978'  (lowercase l vs digit 1)
  'Se ction 302' → 'Section 302'  (space in word)
  'lPC' → 'IPC'  (lowercase l vs uppercase I)
  'Honb|e' → 'Hon'ble'  (pipe char)
  'Rs.l0,000' → 'Rs.10,000'  (l vs 1)
  'petitioner/appel1ant' → 'petitioner/appellant'

Post-processing pipeline:
  1. Legal terminology dictionary lookup (5,000+ legal terms with correct
spelling)
  2. Citation pattern regex correction (AIR, SCC, SCR, All ER patterns)
  3. Section number format standardization
  4. Party name deduplication (OCR may produce multiple variants of same name)
  5. Confidence score per page (low confidence pages shown to user for review)
```

6.4 Session Persistence

The most important user experience feature. Once documents are uploaded, the Case Context persists across all future sessions. The user never re-explains their case.

```
Case workspace URL: nyayarag.in/workspace/{case_id}
Bookmark this URL → your entire case context is always there
All research queries in this workspace automatically use Case Context
Export full research session as PDF with proper citation format
```

SECTION 7 — UI/UX SPECIFICATION

> **Design philosophy: A legal library, not a tech startup. Information density of a law journal. Trust signals of a Supreme Court website. Intelligence of Claude. Transparency of a human expert showing their work.**

7.1 Design System

Typography

Element	Font	Size	Weight	Usage
Body text	Inter	15px	400	All prose, descriptions
Legal citations	Lora (serif)	14px	400 italic	All citation strings — serif signals legal authority
Headings	Inter	18-24px	600	Section headers, answer titles
Code/section numbers	JetBrains Mono	13px	400	Section references, case numbers
UI labels	Inter	12px	500 uppercase	Navigation, metadata labels
Agent log	JetBrains Mono	12px	400	Live agent transparency feed

Color Palette

Token	Value	Usage
--navy-900	#0B1426	Primary background, header, sidebar
--navy-800	#1A2744	Cards, elevated surfaces on dark bg
--cream-50	#F8F6F0	Primary text area background (warm white, not harsh)
--cream-100	#F0EDE4	Secondary surfaces, code blocks
--gold-400	#C9A227	Accent, borders, citation highlights, verified checks
--teal-600	#1D6A72	Primary action, links, interactive elements
--text-primary	#1A1A2E	Main body text
--text-secondary	#4A4A6A	Metadata, secondary info
--green-verified	#1B5E20	Verified citation badge
--yellow-uncertain	#F57F17	Uncertain citation badge
--red-danger	#B71C1C	Reversed/overruled warning
--blue-unverified	#0D47A1	Unverified principle badge

No dark mode in v1. Legal professionals work in controlled environments (courts, offices) — dark mode adds complexity without proportional benefit. Add in v2 based on user demand.

No gradients, no shadows, no glassmorphism, no animations that don't carry information. Every visual element has a semantic reason to exist.

7.2 Screen 1 — Landing Page

Structure:

4. Navigation bar: Logo (NyayaRAG in serif) | Features | Trust | Pricing | Login | Try Free (CTA)
5. Hero: Single bold statement. No carousel, no illustration. Black text on cream background.
6. Trust metric: Live number — '0.0% citation fabrication rate' in large gold text with source note: 'vs 34% Westlaw AI per Stanford 2025'
7. Demo: Embedded 90-second video showing same query in ChatGPT vs NyayaRAG — side by side. ChatGPT hallucinates a case. NyayaRAG shows verified citations with source text.
8. Features: Three columns — Jurisdictions (all 25 HCs + SC + tribunals), Languages (English + 8 Indian languages), Trust (the five verification layers listed)
9. Public evaluation dashboard preview: live metrics table
10. Pricing table
11. Footer: Legal disclaimer (NyayaRAG is a research tool, not legal advice)

What NOT on the landing page: No chatbot demo widget on homepage (sets wrong expectation — this is a research tool not a chatbot). No stock photos of courtrooms. No 'powered by AI' badges.

7.3 Screen 2 — Research Workspace (Core Product)

Layout: Three-panel, resizable

LEFT SIDEBAR (280px, dark navy background):

- User's case files (uploaded documents listed by name)
- Case Context summary (parties, charges, stage — always visible)
- Research history (chronological list of past queries in this workspace)
- Saved answers (starred responses)
- New Case button
- Settings (jurisdiction, language preference)

CENTER PANEL (flexible, cream background):

- Query input at top: single line, with jurisdiction chips below it

- Practice area chips: Criminal / Civil / Constitutional / Family / Tax / Corporate / Labour / Property
- Real-time query understanding: 'Detected: Criminal law, Section 302 IPC, Maharashtra HC' — user can correct
- Research process display (when query running — see Section 7.4)
- Structured answer output (see Section 7.5)
- Iteration bar at bottom: 'Refine this answer...' input for follow-up

RIGHT PANEL (320px, cream background):

- Source viewer — always open
- Shows actual source text when any citation is hovered/clicked
- Scrolls to exact relevant paragraph
- Relevant text highlighted in gold
- Full judgment metadata: court, date, bench, citation, binding status
- 'View full judgment' link to original source

7.4 The Live Research Process Display — Trust Building

This is the most important UI element. When a query runs, the center panel shows what the system is doing in real time. Not a spinner — actual process transparency.

Visual design: monospace font on dark navy panel. Each step appears as it completes. Steps that are running show a pulsing dot. Completed steps show a checkmark. The whole panel fades out when generation begins.

```
Example display for query: 'What are bail conditions for Section 302?'

[●] Analyzing query...
[✓] Detected: Criminal law — Section 302 IPC / BNS Section 101
[✓] Jurisdiction: All India (no specific court selected)
[✓] Query type: Statutory + case law (bail jurisprudence)
[●] Building search strategy...
[✓] Checking BNS/BNSS 2023 equivalents: IPC 302 → BNS 101
[✓] Selected pipeline: Hybrid RAG + CRAG validation
[●] Searching Supreme Court judgments...
[✓] Found 847 judgments citing S.302 + bail
[●] Validating statute currency...
[✓] Section 302 IPC: in force until July 1, 2024
[✓] BNS Section 101: in force from July 1, 2024 (current law)
[●] Reranking by relevance...
[✓] 8 Supreme Court + 4 High Court judgments selected
[●] Validating CRAG score...
[✓] Retrieval confidence: 0.94 (high)
[●] Checking appeal chains...
[✓] All 12 cited cases: final authority confirmed
```

```
[●] Verifying citations...
[✓] 8/8 citations: exist and support claims
[●] Running Self-RAG verification...
[✓] No unsupported claims detected
[✓] Confidence: 96.8%  ▸ Generating answer...
```

7.5 Structured Answer Output

Not a chat bubble. A structured legal document with clear sections.

```
LEGAL POSITION
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
[Answer paragraph — every sentence has inline citation marker]
Section 302 IPC (now BNS Section 101) is a non-bailable offence.[1]
However, courts have held that bail may be granted in exceptional
circumstances including prolonged incarceration pending trial.[2]

APPLICABLE LAW
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
• BNS Section 101 [CURRENT LAW ✓] — Murder — non-bailable
  [IPC Section 302 — SUPERSEDED by BNS from July 1, 2024]
• BNSS Section 480 — Bail provisions for non-bailable offences

KEY CASES — BINDING AUTHORITY
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
①Gudikanti Narasimhulu v PP, (1978) 1 SCC 240  [VERIFIED ✓]
  [BINDING — Supreme Court] Bail is a matter of liberty;
  poverty should not be a reason for pre-trial detention.
  Click to view source →

②Sanjay Chandra v CBI, (2012) 1 SCC 40  [VERIFIED ✓]
  [BINDING — Supreme Court] Bail is rule, jail is exception;
  flight risk, tampering, repeat offence are key considerations.
  Click to view source →

VERIFICATION STATUS
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
[✓] 8/8 citations verified — exist and support claims
[✓] Current law: BNS Section 101 (post July 2024)
[✓] No overruled cases cited
[✓] Confidence score: 96.8%
```

7.6 Citation Color Coding

Visual	Meaning	User Action

Green ✓ badge	Citation exists AND source supports the claim — fully verified	Click to see source text highlighted
Yellow ⚠ badge	Citation exists, partial support — source text shown automatically	Review shown source and judge yourself
Blue ◆ flag	Legal principle identified, specific case not in corpus	Text says 'Requires independent verification'
Red ✗ banner	Cited case was reversed or overruled	Reversal shown instead — cannot cite the reversed case
Gray ~ badge	Case is persuasive authority only (not binding in selected jurisdiction)	Labeled 'PERSUASIVE — [jurisdiction]'

7.7 Citation Graph View (Toggle)

Toggle button in answer header: 'Show citation network'

Renders an interactive D3.js force-directed graph:

- Center node: user's query
- Primary nodes: directly cited cases (larger circles)
- Secondary nodes: cases those cases cite (smaller)
- Node color: gold = Supreme Court, blue = High Court, gray = tribunal
- Edge style: solid = follows, dashed = distinguishes, red = overrules
- Edge thickness: citation frequency
- Click any node: right panel jumps to that judgment
- Hover any node: tooltip shows court, date, ratio_decidendi

7.8 Agent Transparency Drawer

Bottom drawer, opened by clicking 'Show research process' button in answer footer.

Shows complete agent activity log from AgenticRAGState.agent_log:

```
Agent Activity Log — Query #1204
────────────────────────────────────────────────

14:22:01  QueryRouter        Classified: MULTI_HOP_DOCTRINE
14:22:01  GraphRAGAgent      Loading right to privacy concept cluster
14:22:02  GraphRAGAgent      Anchor: Kharak Singh v UP (1963)
14:22:03  GraphRAGAgent      Traversal depth 1: 8 judgments
14:22:04  GraphRAGAgent      Traversal depth 2: 23 judgments
14:22:05  GraphRAGAgent      Overruling detected: A.K. Gopalan overruled by
Maneka Gandhi
14:22:05  GraphRAGAgent      Excluded: A.K. Gopalan (1950) — overruled
14:22:06  CRAGValidator      Score: 0.94 — PROCEED
14:22:06  AppealValidator    3 cases checked — all final authority
14:22:07  CitationVerifier   5/5 citations: existence verified
14:22:08  MisgroundingCheck  5/5 claims: entailment verified (min 0.89)
14:22:08  TemporalChecker    All statutes: currently in force
14:22:09  SelfRAGVerifier    No unsupported claims
```

```
14:22:09  OutputFormatter    Building structured response
──────────────────────────────────────────────────────────
Total: 8.1 seconds | Citations verified: 5/5 | Confidence: 97.4%
```

7.9 Multilingual Integration

Clean integration — not a mess. One language selector in sidebar. That's it.

Language	Code	Input	Output
English	en	Direct — no translation	Direct — primary language
Hindi	hi	IndicTrans2 → English internally	English answer + Hindi explanation
Tamil	ta	IndicTrans2 → English internally	English answer + Tamil explanation
Telugu	te	IndicTrans2 → English internally	English answer + Telugu explanation
Kannada	kn	IndicTrans2 → English internally	English answer + Kannada explanation
Malayalam	ml	IndicTrans2 → English internally	English answer + Malayalam explanation
Marathi	mr	IndicTrans2 → English internally	English answer + Marathi explanation
Bengali	bn	IndicTrans2 → English internally	English answer + Bengali explanation
Gujarati	gu	IndicTrans2 → English internally	English answer + Gujarati explanation

Citations always remain in English (standard Indian legal format). The legal reasoning and explanation is in the user's chosen language. Case names shown in English with transliteration in regional script.

Translation model: IndicTrans2 by IIT Madras — completely free, open-source, best-in-class for Indian languages. Self-hosted, zero API cost.

SECTION 8 — COMPLETE TECHNICAL ARCHITECTURE

8.1 System Architecture Overview

Layer	Technology	Why This Choice
Frontend	Next.js 14 (App Router) + TypeScript + Tailwind	App Router streaming for real-time process display. SSR for fast initial load. Tailwind for custom design system.
Backend API	FastAPI (Python)	Async-first, perfect for streaming LLM responses. Python for ML pipeline integration. Pydantic for type safety.
LLM	Claude claude-sonnet-4-6 (primary) + Claude claude-opus-4-6 (verification)	Sonnet for generation (fast, cheap). Opus for the misgrounding checker and Self-RAG verification (needs strongest reasoning).
Vector DB	Qdrant (self-hosted)	Open-source, free, best metadata filtering in class. Critical for jurisdiction/date/court filtering during retrieval.
Graph DB	Neo4j Community (self-hosted)	For citation graph — native graph traversal. Free tier sufficient for our graph size.
Relational DB	PostgreSQL 15	User accounts, case contexts, query history, billing, evaluation logs.
Cache	Redis	Query result caching (same query = cached response). Citation existence cache (verified citations don't re-verify). Session state.
Embeddings	BGE-M3 (self-hosted via HuggingFace)	Free, multilingual, multi-functional. Runs on CPU for small queries, GPU for batch ingestion.
OCR	Tesseract 5 + TrOCR (for handwriting)	Free, powerful. Custom post-processing for Indian legal terminology.
Translation	IndicTrans2 (self-hosted)	Free, best for Indian languages. IIT Madras model.
Task Queue	Celery + Redis	Background jobs: daily corpus updates, evaluation runs, document processing.
Auth	Clerk	15-minute integration, handles email + social login. Free tier generous.
Payments	Razorpay	India-native, UPI support, lowest Indian market fees.
Hosting	Railway.app or Render (MVP) → AWS/GCP (scale)	Railway free tier for MVP. Migrate when revenue justifies.
CDN	Cloudflare	Free tier. Image serving, DDoS protection, Indian edge nodes.

8.2 Repository Structure

```
nyayarag/
├── frontend/                    # Next.js 14 app
```

```
│   ├── app/
│   │   ├── (auth)/              # login, signup
│   │   ├── workspace/[case_id]/ # main research UI
│   │   ├── trust/               # public evaluation dashboard
│   │   └── api/                 # Next.js API routes (auth, webhooks)
│   ├── components/
│   │   ├── research/            # QueryInput, ProcessDisplay, AnswerPanel
│   │   ├── citation/            # CitationBadge, SourceViewer, CitationGraph
│   │   ├── agents/              # AgentLogDrawer, AgentStatus
│   │   ├── upload/              # DocumentUpload, OCRPreview, CaseContext
│   │   └── ui/                  # design system components (no shadcn
defaults)
│   └── lib/                     # API clients, types, hooks
│
├── backend/                     # FastAPI
│   ├── api/
│   │   ├── routes/              # query, upload, workspace, auth, eval
│   │   └── streaming/           # SSE endpoints for real-time updates
│   ├── rag/
│   │   ├── router.py            # QueryRouter
│   │   ├── pipelines/
│   │   │   ├── hybrid.py        # HybridRAGPipeline
│   │   │   ├── graph.py         # GraphRAGPipeline
│   │   │   ├── crag.py          # CRAGValidator
│   │   │   ├── hyde.py          # HyDEPipeline
│   │   │   └── agentic.py       # AgenticRAGPipeline (LangGraph)
│   │   └── verification/
│   │       ├── citation.py      # Type 1: existence verifier
│   │       ├── misgrounding.py  # Type 2: semantic entailment checker
│   │       ├── appeal_chain.py  # Appeal hierarchy validator
│   │       └── self_rag.py      # Final verification pass
│   ├── ingestion/
│   │   ├── scrapers/            # SC, HC, eCourts, indiacode scrapers
│   │   ├── parsers/             # PDF, OCR, Word parsers
│   │   ├── extractors/          # NER, citation graph, metadata
│   │   └── validators/          # validity checker, amendment tracker
│   ├── evaluation/              # all 25 metrics implementation
│   ├── models/                  # all dataclasses and Pydantic models
│   └── tasks/                   # Celery background tasks
│
├── data/
│   ├── corpus/                  # raw scraped documents
│   ├── processed/               # chunked, embedded, indexed
│   ├── graphs/                  # citation graph exports
│   └── evaluation/              # test sets, annotations
│
└── infrastructure/
    ├── docker-compose.yml       # Qdrant + Neo4j + Redis + PostgreSQL
    ├── scripts/                 # corpus update scripts
```

```
└── evaluation/              # weekly eval runner
```

8.3 API Endpoints — Complete List

Method	Endpoint	Description	Auth
POST	/api/query	Submit research query — returns SSE stream	Required
GET	/api/query/{id}/stream	SSE stream: process steps + answer chunks	Required
POST	/api/workspace	Create new case workspace	Required
GET	/api/workspace/{case_id}	Get workspace with case context	Required
POST	/api/workspace/{case_id}/upload	Upload case documents	Required
GET	/api/workspace/{case_id}/history	Query history for this workspace	Required
GET	/api/citation/{doc_id}/verify	Verify a specific citation	Required
GET	/api/citation/{doc_id}/source	Get source text for citation	Required
GET	/api/citation/{doc_id}/appeal-chain	Get full appeal history	Required
GET	/api/judgment/{doc_id}	Get full judgment metadata	Required
GET	/api/statute/{act_id}/section/{num}	Get current section text	Public
GET	/api/evaluation/public	Public evaluation dashboard data	Public
POST	/api/auth/signup	User registration	Public
POST	/api/billing/subscribe	Create Razorpay subscription	Required
GET	/api/health	System health check	Public

8.4 Streaming Architecture — How Real-Time Display Works

The research process display requires real-time updates from backend to frontend. Implementation: Server-Sent Events (SSE).

```
Backend (FastAPI):
@app.get('/api/query/{query_id}/stream')
async def stream_query(query_id: str):
    async def generate():
        async for event in query_executor.run(query_id):
            yield f'data: {event.to_json()}\n\n'
    return StreamingResponse(generate(), media_type='text/event-stream')

Event types streamed:
  {type: 'STEP_START', step: 'Analyzing query...'}
  {type: 'STEP_COMPLETE', step: 'Query analyzed', data: {...}}
  {type: 'STEP_ERROR', step: '...', error: '...'}
  {type: 'AGENT_LOG', agent: 'GraphRAGAgent', message: '...', ts: '...'}
```

```
  {type: 'TOKEN', token: 'Section'}  # streaming answer tokens
  {type: 'CITATION_RESOLVED', placeholder: '...', citation: '...', status:
'VERIFIED'}
  {type: 'COMPLETE', confidence: 0.968, metrics: {...}}

Frontend (Next.js):
const eventSource = new EventSource('/api/query/{id}/stream')
eventSource.onmessage = (e) => dispatch(handleEvent(JSON.parse(e.data)))
```

8.5 Qdrant Collections

Collection	Contents	Chunk Size	Embedding Model	Key Filters
sc_judgments	All Supreme Court judgments	512 tokens, sentence-boundary	BGE-M3	date, bench_size, validity
hc_judgments	All 25 HC judgments	512 tokens	BGE-M3	court, state, date, validity
statutes	All Act sections — current text	256 tokens (section=1 chunk)	BGE-M3	act, section_num, validity, state
constitution	All articles + amendment history	Article=1 chunk	BGE-M3	article_num, amendment_num
tribunal_orders	NCLT, ITAT, NGT etc	512 tokens	BGE-M3	tribunal, date, validity
lc_reports	Law Commission reports 1-280	512 tokens	BGE-M3	report_num, topic
doctrine_clusters	Pre-computed doctrine summaries	1024 tokens	BGE-M3	doctrine_name, area

SECTION 9 — MONETIZATION & GO-TO-MARKET

9.1 Pricing Tiers

Tier	Price	Queries	Features	Target User
Free	₹0/month	20/day	SC + all HCs, basic citation verification, no upload, no export	Law students, junior advocates, public users
Advocate Pro	₹799/month	Unlimited	All courts + tribunals + all statutes, document upload (5 cases), citation graph, PDF export with Indian citation format, 1-year history	Solo practicing advocates
Chamber Pro	₹2,499/month	Unlimited (5 seats)	Everything in Pro + shared workspaces + collaborative research + API access + priority support	Law firms, chambers, legal departments
Law School	₹9,999/year per institution	Unlimited students	Academic format, moot court tools, student management, bulk account creation	NLUs, law colleges, universities
Enterprise	Custom	Unlimited	White-label, custom training on proprietary case files, dedicated instance, SLA	Large law firms, corporates, government

9.2 Revenue Projections

Month	Free Users	Pro Users	Chamber Accounts	MRR
Month 1 (launch)	500	50	5	₹52,450
Month 3	2,000	200	20	₹2,09,800
Month 6	8,000	800	100	₹8,89,900
Month 12	25,000	2,000	300	₹23,49,700

9.3 Distribution Strategy

Day 1 — Organic

- Post demo video in Bar Council of India WhatsApp groups (every state has active groups)
- Post on Legal India Twitter/X — lawyers are extremely active here
- Submit to eCourts Innovation Challenge — government program, legitimizes the product
- Post on r/india and r/LegalAdviceIndia — large communities with legal questions

- Reach out to 5 NLU professors directly — they influence student adoption

The Demo Video — Primary Marketing Asset

One 3-minute screen recording:

12. Ask ChatGPT: 'What are the landmark cases on right to privacy in India?' — it hallucinates case names
13. Ask the same question on a competitor — shows citations but user cannot verify them
14. Ask on NyayaRAG — shows citation graph, every case verified, source text shown, confidence 97.4%
15. Try to make NyayaRAG hallucinate — show it surfaces the UNVERIFIED badge instead

This video, posted on Legal India Twitter the day of launch, will go viral in the legal community. The Supreme Court's February 2026 ruling makes every lawyer acutely aware of hallucination risk. You are showing the solution.

SECTION 10 — BUILD PHASES — CODEX EXECUTION PLAN

> **This is the Codex prompt sequence. Each phase has a clear deliverable and a test that proves it works before moving to the next phase.**

Phase 1 — Data Pipeline (Week 1-2)

Build order:

16. India Kanoon scraper — pulls 15M+ documents via free API. Store in PostgreSQL with all metadata fields from Section 2.2.1.
17. indiacode.nic.in scraper — all Central Acts, all sections, amendment history.
18. Constitution parser — all 395 articles + 106 amendments + Constituent Assembly Debates.
19. BNS/BNSS/BSA ingestion — the new criminal codes that replaced IPC/CrPC/Evidence Act from July 2024.
20. Validity checker — mark every IPC/CrPC/Evidence Act section with its BNS/BNSS/BSA equivalent.
21. Citation extractor — parse every judgment, extract all case citations, build raw citation list.
22. Appeal chain builder — for every cited case, find its appeal history. Build AppealNode chain.
23. Citation graph builder — convert citation list to Neo4j graph with typed edges.
24. Chunking pipeline — split judgments into 512-token chunks at sentence boundaries.
25. Embedding pipeline — BGE-M3 embed all chunks, store in Qdrant with metadata.

Phase 1 test:

```
python test_phase1.py
Expected output:
  ✓ India Kanoon: 1,000,000+ documents indexed
  ✓ All Central Acts: 900+ acts, 50,000+ sections
  ✓ Constitution: 395 articles + amendments
  ✓ Citation graph: 500,000+ edges
  ✓ Appeal chains: 100,000+ cases resolved
  ✓ Qdrant: embeddings indexed
  ✓ Validity: all IPC sections mapped to BNS
```

Phase 2 — RAG Pipelines (Week 3-4)

Build order:

26. QueryRouter — classification logic for all 7 query types.

27. HybridRAGPipeline — BM25 + dense retrieval + RRF + cross-encoder reranking + authority ranking.
28. GraphRAGPipeline — citation graph traversal, doctrinal timeline construction.
29. CRAGValidator — relevance scoring + temporal validation layer.
30. HyDEPipeline — hypothetical generation + retrieval.
31. AgenticRAGPipeline — LangGraph graph with all 7 agents.

Phase 2 test:

```
Run 50 manually crafted queries covering all types.
Measure: retrieval precision@5 > 0.80 on all types.
```

Phase 3 — Zero Hallucination Layer (Week 5)

Build order:

32. Constrained generation prompt — placeholder-based citation system.
33. Citation resolver — existence verification against corpus.
34. Misgrounding detector — semantic similarity + NLI entailment.
35. Appeal chain validator — prevents citing reversed judgments.
36. Self-RAG verifier — final pass over complete output.
37. Output builder — constructs structured response with CitationStatus per claim.

Phase 3 test:

```
Run 200 queries. Manually verify 50 randomly selected responses.
Expected: citation existence rate = 100%. Citation accuracy > 98%.
Run Stanford hallucination test set (adapted for Indian law).
```

Phase 4 — Evaluation Framework (Week 6)

Build order:

38. RAGAS integration — faithfulness, answer relevancy, context precision/recall.
39. DeepEval integration — hallucination score, contextual metrics.
40. All 8 India-legal-specific metrics implemented.
41. Latency benchmarking — p50/p95/p99 across all pipeline types.
42. Automated evaluation runner — runs weekly, writes to PostgreSQL.
43. Public dashboard API — serves live metrics to frontend.

Phase 5 — Document Upload (Week 7)

Build order:

44. Typed PDF pipeline — pdfplumber + legal text cleanup.
45. Scanned PDF pipeline — Tesseract + post-processing dictionary.
46. Photo pipeline — image preprocessing + same OCR.
47. Word document pipeline — python-docx.
48. Page classifier — typed vs scanned per page.
49. Legal NER — extract parties, charges, dates, sections.
50. CaseContext builder — assemble from all document extractions.
51. Session persistence — case_id → CaseContext stored in PostgreSQL.

Phase 6 — Frontend (Week 8-9)

Build order:

52. Design system — typography, colors, component primitives (no shadcn defaults used as-is — customize everything).
53. Landing page — hero, trust metric, demo video embed, evaluation dashboard preview, pricing.
54. Auth flow — Clerk integration, signup/login.
55. Workspace layout — three-panel resizable layout.
56. Query input — with jurisdiction chips, practice area chips, real-time query understanding.
57. Research process display — SSE consumer, step-by-step animation.
58. Answer output — structured sections, inline citation badges, hover-to-source.
59. Source viewer panel — loads source text, highlights relevant paragraph.
60. Citation graph — D3.js force-directed graph.
61. Agent transparency drawer — full activity log.
62. Document upload — drag-drop, OCR progress, CaseContext display.
63. Research history — sidebar with past queries.
64. Export — PDF with proper Indian citation format.
65. Multilingual — language selector, IndicTrans2 integration.

Phase 7 — Billing + Launch (Week 10)

66. Razorpay subscription integration.
67. Usage tracking — queries per user per day.
68. Free tier enforcement — 20 queries/day rate limit.
69. Billing portal — upgrade, downgrade, invoice history.
70. Public evaluation dashboard live at /trust.
71. Demo video recorded and edited.
72. Launch: Legal India Twitter + Bar Council WhatsApp groups + Product Hunt.

SECTION 11 — CODEX EXECUTION INSTRUCTIONS

This section is written specifically for Codex (GPT-5). Each prompt below corresponds to one build unit. Run them sequentially. Never skip a test before proceeding.

11.1 How to Use This Document with Codex

73. Give Codex this entire document as context at the start of each session.
74. Specify the exact phase and build unit you are working on.
75. Tell Codex: 'Do not simplify. Build exactly as specified. Use the data models in Section 2. Follow the pipeline architecture in Section 3. Implement all verification layers in Section 4.'
76. After each unit: run the specified test. If test fails — debug before proceeding.
77. Never let Codex skip the verification layers. If Codex suggests a simpler approach that removes the citation verifier or misgrounding checker — reject it. Those are non-negotiable.

11.2 Master Codex Prompt Template

```
You are building NyayaRAG — the Indian legal research system described in
this technical specification document. You are now working on:

[PHASE]: Phase {N} — {Phase Name}
[UNIT]: {Specific build unit from the phase}

Critical constraints that must never be simplified away:
1. Every citation must go through the existence verifier before appearing in
output
2. Every claim must go through the misgrounding checker (Section 4.2)
3. Every cited judgment must have its appeal chain validated (Section 4.3)
4. Every statute must have its temporal validity checked (Section 2.4)
5. The Self-RAG verifier (Section 4.4) must run on every response

Use exactly the data models defined in Section 2.
Use exactly the pipeline architecture defined in Section 3.
Use the tech stack specified in Section 8.1.

Build this unit completely. Include tests. Do not simplify.
```

11.3 Critical Non-Negotiables for Codex

If Codex suggests any of the following — reject and re-prompt:

- Removing the citation verifier 'to keep things simple'
- Using a single RAG pipeline instead of the router
- Skipping the misgrounding detector
- Using shadcn/ui components without customizing them to the design system
- Using ada-002 embeddings instead of BGE-M3
- Storing case citations as strings without doc_id references
- Using a boolean is_valid instead of the full ValidityStatus enum
- Skipping the appeal chain validator
- Using a loading spinner instead of the streaming process display

---

NyayaRAG — Built for Indian justice. Zero hallucination. Architecturally guaranteed.

End of Technical Specification v1.0